Exam Scheduling Using Genetic Algorithm – Report

Team Membres :

Ahmed Elsawy 2305238

Moamen Mahmoud 2305520

Moataz Mahmoud 2305219

Muhab Ahmed 2305386

Abdelrahman Ehab 2305459

Objective

The goal is to schedule exams for various courses such that:

- No student has overlapping exams.
- Classrooms can accommodate all students.
- Timeslots and classrooms are efficiently used.

A **Genetic Algorithm (GA)** is employed to find a near-optimal schedule under these constraints.

Step-by-Step Description

1. Data Import

The following CSV files are loaded using `pandas`:

- `students.csv`: List of student IDs.
- `courses.csv`: List of course IDs.
- `instructors.csv`: List of instructors.
- `classrooms.csv`: Classroom IDs and capacities.
- `timeslots.csv`: Timeslot IDs.
- `schedule.csv`: Each row indicates a student enrolled in a course taught by an instructor.

2. Data Preprocessingg

From the raw data, the following mappings are created:

- `student_course_map`: Which students are enrolled in which courses.
- `course_student_count`: Number of students per course.
- `course_instructor`: Instructor assigned to each course.
- `classroom_capacities`: Capacity lookup by classroom ID.
- Lists of all entities: `courses`, `students`, `classrooms`, `instructors`, `timeslots`.

These mappings are used for fast constraint checking during optimization.

3. Genetic Algorithm Setup

Each **individual** in the population represents a full exam schedule:

- A list of `(timeslot, classroom)` pairs, one for each course.

Fitness Function

The fitness function penalizes:

- **Student Conflict**: A student has two exams scheduled at the same time.
- **Overcapacity**: Number of enrolled students exceeds classroom capacity.

The penalty is returned as a **negative value** (since DEAP maximizes fitness).

GA Configuration (using DEAP)

- **Individual**: `[(timeslot, classroom), ...]` for each course
- **Initialization**: Randomly assign a timeslot and classroom for each course
- **Selection**: Tournament selection (size 3)
- **Crossover**: Two-point crossover
- **Mutation**: 20% chance to reassign a timeslot/classroom for each gene (course)
- **Population**: 100 individuals
- **Generations**: 50

4. Genetic Algorithm Execution

```
for gen in range(NGEN):
    ...
```

For each generation:

- **Selection**: Choose parents using tournament selection.
- **Crossover**: Recombine parent schedules.
- **Mutation**: Apply small random changes.
- **Evaluation**: Assign fitness based on penalties.
- **Replacement**: Update the population with new individuals.

5. Extracting the Best Schedule

After 50 generations, the best schedule is selected using:

```
best = tools.selBest(population, 1)[0]
```

Each course is then mapped to its assigned `(timeslot, classroom)` and displayed in a DataFrame.

Expected Output

The final output is a table that looks like this:

course_id	timeslot	classroom
C1	T1	R1
C2	T2	R2
...	...	...

This schedule minimizes student conflicts and ensures room capacities are not violated.

Conclusion

This project successfully demonstrates the use of Genetic Algorithms for solving the exam scheduling problem, which is inherently complex due to multiple constraints such as student overlap and classroom capacity. By modeling the problem appropriately and tuning the algorithm's parameters, a feasible and efficient schedule was generated.

While the GA may not always find the perfect schedule, it offers a highly adaptable and practical approach that performs well for real-world educational institutions. With additional refinement and integration, this solution can serve as the backbone for an automated exam scheduling system.